

A Standards-Checked X3D 4.0 Editor for the Browser

with Direct Manipulation, Format Conversion and an AI Co-Author

Web3D/Metaverse Tools Competition summary for *X3D Copilot*

Alberto Jaspe-Villanueva, King Abdullah University of Science and Technology (KAUST), alberto.jaspe@kaust.edu.sa

Live tool: <https://albertojaspe.net/x3d-copilot/>

Source (MIT): <https://github.com/ajaspe/x3d-copilot>

Demonstration video (4:25): <https://youtu.be/hkA0HNPSj7Y>

Tool name: X3D Copilot (new). *Type:* editor, validator, converter (importer/exporter). *Domains:* education, content creation, research prototyping, quality assurance of X3D assets.

Open source: yes, MIT. *Note:* the AI co-author needs the user's own Gemini or Claude key (Settings); all other functions work without one.

Abstract

X3D Copilot is a web application for creating, validating and converting X3D scenes in the current version of the standard, X3D 4.0, with no installation. It provides a live X_{ITE} view, a source editor with autocompletion derived from the X3D specification, direct manipulation of objects through a gizmo that is itself built from X3D sensor nodes, import of glTF, OBJ, STL, PLY, SVG and VRML as editable X3D, and export to X3D XML, Classic VRML, JSON and PNG. Every change, whether typed, dragged or generated, is checked by three independent validators: the official X3D 4.0 XML Schema, running in the browser through libxml2 compiled to WebAssembly; a semantic linter of about thirty rules derived from the X3D Unified Object Model; and the X_{ITE} runtime. An optional AI co-author works through the same validators, so scenes it writes or repairs conform to the standard. The tool is open source, covered by 56 automated tests, and deployed as a static site.

Problem solved

X3D authoring today means either a desktop tool or a text editor plus a separate validator, and mistakes are silent: a misspelled field, a ROUTE to a non-existent event, a **Material** placed directly in a **Shape** or an interpolator with mismatched keys produce an empty or wrong scene without explanation. Assets converted from other formats often carry attributes that are not X3D 4.0. And the newest authoring aid, language models, makes these mistakes at scale. X3D Copilot gives students, researchers and content creators a place where the standard itself checks every edit, explains what is wrong in plain language, and where an assistant is forced through the same checks.

How the tool meets the judged functionalities

1. Adherence to X3D standards. The tool is built from the standard's own artefacts. The vendored `x3d-4.0.xsd`, with the Web3D extension schemas, is the schema validator. The X3D Unified Object Model 4.0 is compiled at build time into a database of 260 concrete nodes, 78 abstract types and 17 statements that drives autocompletion, the linter's placement and type rules, and the AI's node lookups. Rendering and conversion use X_{ITE}, an X3D 4.0 browser; imported scenes are normalised to X3D 4.0 (version set, attributes outside 4.0 dropped and reported). Output is plain, portable `.x3d`; the gizmo and selection sensors exist only in the runtime scene graph and never enter the file.

2. User-friendly interface. Three resizable panes:

chat, view, source. Click an object to select it; move, rotate and scale with the gizmo or scrub numbers in the inspector; the text cursor selects nodes in the view and vice versa. Autocompletion lists the nodes a parent accepts and each field's type and default. Issues are graded, attributed to their source, phrased as actions ("**<Material>** goes inside **<Appearance>**"), with did-you-mean suggestions and jump-to-line. Eight example scenes, including one authored entirely through the assistant and one deliberately broken.

3. User configurability. Shading mode (Phong, Gouraud, flat, wireframe, points); issue filters (schema, hints); gizmo modes (move, rotate, scale, uniform lock); selection on or off; pane widths; AI provider, model, reasoning effort and proxy URL; screenshot policy. Settings persist in the browser.

4. Performance. Static site, no server. The schema is fetched and compiled once (about one second) and cached; validation of a typical scene takes tens of milliseconds and runs on every keystroke after a short debounce; rendering is X_{ITE}'s. The only operations that take seconds are calls to the optional model.

5. Extensibility. Validators are independent modules sharing one issue format; the specification database is regenerated from the X3DUOM by one script; AI providers are adapters of about 150 lines behind a common interface; AI tools are JSON-schema definitions with local executors, so new capabilities (HAnim rigging, glTF export) plug in without touching the agent loop. The build is TypeScript with Vite.

6. Error handling. Three layers catch what the others cannot: well-formedness and schema, semantic rules (DEF/USE integrity, ROUTE node, field, access and type checks, container-field placement, SFNode multiplicity, attribute arity and ranges, interpolator consistency, empty groups, missing viewpoint), and runtime errors and warnings from X_{ITE} (unsupported profile, missing texture, bad URL). Schema messages that duplicate a linter finding are demoted to avoid double counting. Import failures, model errors and quota problems are reported in plain language.

7. Advanced features. Animation (TimeSensor, interpolators, sequencers), interaction (touch, plane, cylinder and proximity sensors, event utilities), X3D 4.0 physically based materials, shadows, texture projection, HAnim 2.0 characters, Script nodes, shaders, and the format converters, all through X_{ITE}. The examples exercise nested animation, click behaviours, PBR and HAnim.

8. Test cases. 56 automated tests: schema validation

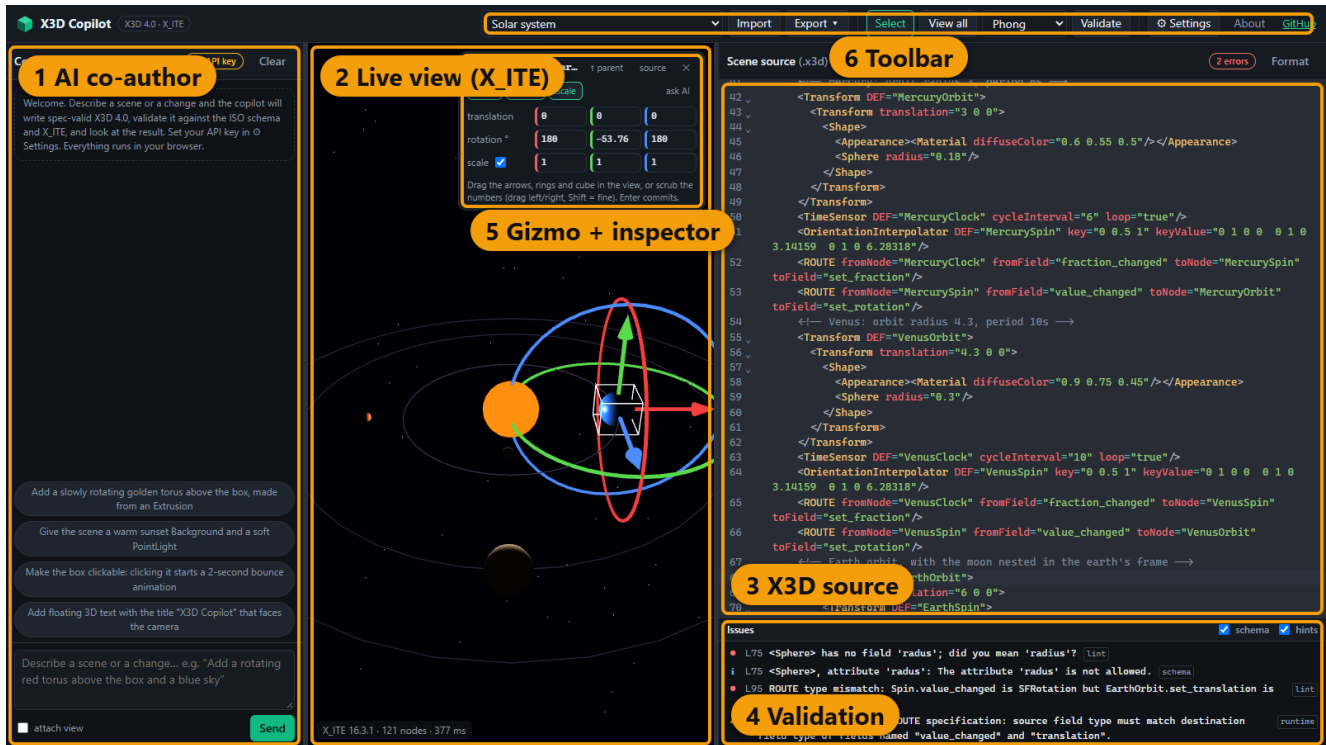


Figure 1: The tool with the solar system example open and the earth selected. Two mistakes were typed into the source (a misspelled **radius**, a ROUTE from an SFRotation output to an SFVec3f input): the validators report both with line numbers and suggestions. (1) AI co-author, optional. (2) Live view rendered by X_ITE, with the transform gizmo on the selected object. (3) X3D source with specification-driven autocompletion and inline diagnostics. (4) Validation issues from schema, linter and runtime, click to jump to the line. (5) Inspector: translation, rotation and scale, synchronised with the source. (6) Toolbar: examples, import, export, selection, view control, shading, validate, settings.

fixtures (valid and invalid), one test per linter rule, the edit-patching tool, the source map and gizmo mathematics, the X3D 4.0 normaliser, and a check that every shipped example is lint-clean and schema-valid while the broken example triggers nine specific rules. Continuous integration runs typecheck, tests and build on every push.

9. Open-source accessibility. MIT license on GitHub, hosted as static files on the author’s site; no accounts or servers beyond the user’s own model key for the optional assistant; runs in any current browser.

The AI co-author

The assistant (Google Gemini or Anthropic Claude, chosen by the user) acts only through tools: read the scene, apply exact patches, replace it, validate, look up nodes in the specification database, and take a screenshot. Every edit returns the validation report, so the model repairs its own mistakes before answering and then checks the render. In the video it builds a studio still life from one sentence and repairs the eleven-error example in a single request. Its steps are shown to the user and can be expanded.

Three typical sessions

A student learning X3D opens the broken example, reads eleven explanations with line numbers, fixes them one by one with autocompletion, and watches the badge turn to “valid X3D 4.0” and the scene render. **A researcher with a gITF model** drops it on the page, gets editable X3D 4.0 source normalised from X_ITE’s importer, adds a Viewpoint and a TouchSensor by hand, and exports Classic VRML for a legacy pipeline. **A content creator** types “a snowman at night with moonlight

shadows”, clicks the snowman to select its hat, scrubs the rotation in the inspector, asks “make his hat jump when clicked”, and saves a scene that passes the schema.

Why this entry

Tool of the Year: a complete, usable X3D 4.0 authoring environment with zero installation that raises conformance through the standard’s own schema and object model, and that is tested, documented and open. **Tool/Pipeline Innovation of the Year:** the first X3D editor, to our knowledge, whose transform gizmo is expressed in X3D sensor nodes, and whose AI assistant is grounded in the ISO object model and closed-loop validated by schema, linter, runtime and its own screenshot.

Built with X_ITE (Holger Seelig, MIT), libxml2-wasm, CodeMirror 6, the Google Gen AI and Anthropic SDKs. X3D schema and X3DUOM © Web3D Consortium. The author thanks the X3D Working Group for maintaining the machine-readable specification artefacts that make this tool possible.